

Decision Log

Version: 1.0

Purpose

This document provides the central index of significant product, architecture, technology and development-process decisions for Trading Platform Pro and its primary application, the Trading Cockpit.

The Decision Log exists to make important project decisions:

- discoverable
- traceable
- reviewable
- version controlled

It does not replace Architecture Decision Records.

Architecture-significant decisions shall be documented in `docs/adr/`.

Decision Model

The project distinguishes between:

```
```text
```

Architecture Decision

↓

Architecture Decision Record

↓

docs/adr/

Product, Technology or Process Decision

↓

Decision Log Entry

↓

docs/decisions/Decision\_Log.md

```
```
```

The Decision Log may reference ADRs.

Do not maintain the same decision independently in both systems.

Architecture Decisions

Use an Architecture Decision Record when a decision has significant and durable architecture impact.

Examples:

- dependency direction
- major capability boundaries
- Domain ownership
- Application workflow ownership
- persistence architecture
- broker integration architecture
- market data architecture
- runtime lifecycle architecture
- AsyncIO integration strategy
- dependency injection strategy
- major cross-capability infrastructure

ADR rules are defined in:

```
```text
```

docs/adr/README.md

```
```
```

Architecture decisions may appear in the Decision Index as references to the owning ADR. The ADR remains the authoritative decision record.

Decision Log Decisions

Use the Decision Log for significant decisions that require durable project context but do not require a dedicated ADR.

Examples:

- primary product direction
- supported Python version
- source quality tooling
- CI technology
- documentation source policy
- development workflow policy
- release process decisions
- repository policy

Do not create Decision Log entries for:

- routine bug fixes
- formatting changes
- local refactoring
- test additions
- small implementation details
- ordinary dependency updates

Product Direction

The Trading Cockpit is the primary product.

Current product and architecture decisions shall optimize for concrete Trading Cockpit requirements.

Product development follows:

```text

Trading Cockpit Requirement

↓

Vertical Product Capability

↓

Application and Domain Behaviour

↓

Required Infrastructure

```

Shared platform capabilities may evolve when multiple concrete product capabilities require them. Do not introduce generic platform infrastructure solely for hypothetical future applications.

Decision Identifier

Decision Log entries use sequential identifiers.

Format:

```text

DEC-NNNN

```

Examples:

```text

DEC-0001

DEC-0002

DEC-0003

```

Identifiers shall not be reused.

A superseded or deprecated decision remains part of the project history.

ADR identifiers use the separate format:

```
```text
ADR-NNNN
```
```

Do not assign both a DEC identifier and an ADR identifier to the same authoritative decision record.

Decision Status

Decision Log entries use one of the following statuses:

```
```text
Proposed
Accepted
Superseded
Deprecated
Rejected
```
```

Proposed

The decision is under consideration.

A Proposed decision shall not be treated as an accepted project contract unless an explicitly approved prototype requires it.

Accepted

The decision is approved and active.

Current project documentation shall remain consistent with accepted decisions.

Superseded

The decision has been replaced by another decision or ADR.

The replacing decision shall be identified.

Example:

```
```text
Status: Superseded
Superseded by: DEC-0008
```
```

or:

```
```text
Status: Superseded
Superseded by: ADR-0005
```
```

Do not delete superseded decisions.

Deprecated

The decision is no longer recommended or is being phased out.

Use Deprecated when migration is incomplete and the decision remains relevant to current project state.

The entry shall explain the migration context.

Rejected

The proposed decision was evaluated and explicitly not selected.

Rejected decisions should be preserved only when the rationale provides durable project context.
Do not record every informal alternative as a rejected decision.

Decision Entry Structure

Each Decision Log entry shall use the following structure:

```markdown

### **DEC-NNNN: Decision Title**

Date: YYYY-MM-DD

Status: Proposed

#### ***Context***

Describe the concrete product, technology or process requirement.

#### ***Decision***

Describe the selected decision explicitly.

#### ***Rationale***

Explain why the decision was selected.

#### ***Alternatives Considered***

Describe materially relevant alternatives.

#### ***Consequences***

Positive:

- ...

Negative:

- ...

#### ***Impact***

Identify affected project areas.

#### ***Related Documents***

- ...

```

Additional sections may be added when required.

Do not remove the core decision context.

Context

The Context section shall describe the concrete problem or requirement.

Good context identifies:

- current situation
- concrete requirement
- current limitation
- affected project area

Do not use speculative future scalability as the only context unless a concrete approved requirement exists.

Decision

The Decision section shall be explicit.

Prefer:

```text

The Trading Cockpit shall be the primary product and architecture decisions shall optimize for concrete Trading Cockpit requirements.

```

Avoid:

```text

We should probably focus more on the application.

```

The selected direction shall be unambiguous.

Rationale

The Rationale section explains why the decision was selected.

Relevant criteria may include:

- product requirement
- architecture consistency
- maintainability
- testability
- operational safety
- development efficiency
- migration cost
- real consumers

Avoid generic claims such as:

```text

More scalable

More enterprise-ready

More modern

```

without concrete project-specific reasoning.

Alternatives Considered

Document materially relevant alternatives.

An alternative should have been realistic enough to affect the decision.

Do not invent artificial alternatives solely to complete the template.

For each alternative explain why it was not selected.

Architecture-significant alternatives belong primarily in the owning ADR.

Consequences

Document both positive and negative consequences.

Positive consequences may include:

- clearer product focus
- reduced complexity
- stronger ownership
- improved testability
- simpler development workflow

Negative consequences may include:

- migration effort
- reduced generic reuse
- stronger process requirements

- additional validation
 - explicit documentation maintenance
- Do not describe significant decisions as having only benefits.

Impact

Identify concrete affected project areas.

Examples:

- Product
- Architecture
- Domain
- Application
- Infrastructure
- Presentation
- Runtime
- Persistence
- Configuration
- Testing
- Documentation
- CI/CD
- Development Workflow

Do not list unaffected areas solely to make the entry appear comprehensive.

Decision Index

The Decision Index is the central discovery mechanism for accepted and historical decisions.

| ID | Title | Type | Status | Authoritative Record |

|----|-----|-----|-----|-----|

| DEC-0001 | Trading Cockpit as Primary Product | Product | Accepted | Decision Log |

| DEC-0002 | Python 3.13 | Technology | Accepted | Decision Log |

| DEC-0003 | Ruff as Formatter and Linter | Technology | Accepted | Decision Log |

| DEC-0004 | GitHub Actions for CI | Technology | Accepted | Decision Log |

| DEC-0005 | Markdown as Documentation Source of Truth | Process | Accepted | Decision Log |

| DEC-0006 | Documentation Generation Before Commit | Process | Accepted | Decision Log |

| DEC-0007 | Dedicated Development Branches | Process | Accepted | Decision Log |

Architecture decisions shall be added to the index only when an owning ADR exists.

Example:

```
```text
```

| ADR-0005 | Use Vertical Product Slices | Architecture | Accepted |

docs/adr/0005-use-vertical-product-slices.md |

```
```
```

Do not list architecture slogans such as:

```
```text
```

Clean Architecture

Domain-Driven Design

Event-Driven Architecture

```
```
```

as accepted project decisions unless a concrete project-specific ADR defines the actual decision and its consequences.

Initial Decision Entries

The following entries establish the current project decision baseline.

DEC-0001: Trading Cockpit as Primary Product

Date: 2026-07-01

Status: Accepted

Context

Trading Platform Pro was previously described primarily as a generic modular platform supporting future trading applications.

The current product requirement is to develop the Trading Cockpit as the primary concrete product. Generic platform-first development introduced a risk of speculative abstractions without demonstrated consumers.

Decision

The Trading Cockpit shall be the primary product.

Architecture and development decisions shall optimize for concrete Trading Cockpit requirements.

Shared platform capabilities may evolve when multiple concrete product capabilities require them.

Rationale

The decision:

- aligns architecture with the current product
- reduces speculative infrastructure
- supports vertical product capabilities
- keeps shared abstractions demand-driven

Alternatives Considered

Generic platform-first development was considered.

It was not selected because future application requirements are not sufficiently concrete to justify platform abstractions as the primary architecture driver.

Consequences

Positive:

- clearer product direction
- reduced speculative architecture
- stronger capability ownership
- simpler prioritization

Negative:

- future applications may require later extraction of shared capabilities
- some generic reuse is intentionally deferred

Impact

- Product
- Architecture
- Development Workflow
- Documentation

Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md

DEC-0002: Python 3.13

Date: 2026-07-01

Status: Accepted

Context

The project requires one explicit supported Python runtime for development, testing and CI.

Decision

Python 3.13 shall be the supported Python version.

Rationale

A single explicit runtime reduces environment ambiguity and keeps local development and CI aligned.

Alternatives Considered

Supporting multiple Python versions was considered.

It was not selected because the current product does not require a multi-version compatibility contract.

Consequences

Positive:

- consistent runtime
- simpler CI
- simpler dependency validation

Negative:

- consumers require Python 3.13
- runtime upgrades require an explicit project decision

Impact

- Technology
- Development
- CI/CD

Related Documents

- Technical_Specifications.md
- Coding_Standards.md
- CI_CD.md

DEC-0003: Ruff as Formatter and Linter

Date: 2026-07-01

Status: Accepted

Context

The project requires consistent source formatting and static source quality validation.

Decision

Ruff shall be used for linting and formatting verification.

Required checks include:


```
```bash
ruff check .
ruff format --check .
```
```

Rationale

Using one source quality tool reduces tooling complexity and keeps local development and CI verification aligned.

Alternatives Considered

Separate linting and formatting toolchains were considered.
They were not selected because the current project does not require separate tools.

Consequences

Positive:

- consistent source quality workflow
- reduced tooling complexity
- fast local verification

Negative:

- project formatting behaviour depends on Ruff
- Ruff configuration changes may affect broad source areas

Impact

- Technology
- Development Workflow
- CI/CD

Related Documents

- Coding_Standards.md
- Development_Guidelines.md
- CI_CD.md

DEC-0004: GitHub Actions for CI

Date: 2026-07-01

Status: Accepted

Context

The repository requires automated quality gates for integration changes.

Decision

GitHub Actions shall provide the project CI workflow.
Only implemented and configured quality gates shall be described as actively enforced.

Rationale

GitHub Actions integrates directly with the repository workflow and supports automated source, test and documentation validation.

Alternatives Considered

External CI platforms were considered.

They were not selected because no concrete project requirement requires an additional CI platform.

Consequences

Positive:

- repository-integrated CI
- automated quality gates
- visible workflow results

Negative:

- CI configuration depends on GitHub Actions
- provider workflow changes may require CI maintenance

Impact

- Technology
- CI/CD
- Development Workflow

Related Documents

- CI_CD.md
- Git_Workflow.md
- Development_Guidelines.md

DEC-0005: Markdown as Documentation Source of Truth

Date: 2026-07-01

Status: Accepted

Context

The project generates DOCX and PDF documentation from Markdown.

Maintaining multiple editable documentation formats would create synchronization risk.

Decision

Markdown under:

```
``text
docs/
```
```

shall be the documentation source of truth.

Generated DOCX and PDF files are derived artifacts and shall not be edited manually.

### ***Rationale***

One editable documentation source reduces inconsistency and supports automated generation.

### ***Alternatives Considered***

Maintaining Markdown, DOCX and PDF as independent editable sources was considered.

It was not selected because synchronization would be manual and error-prone.

### ***Consequences***

Positive:

- one documentation source
- deterministic generation workflow
- simpler review

Negative:

- documentation generation tooling becomes part of the workflow
- generated format corrections must originate from Markdown or generation logic

### ***Impact***

- Documentation
- Development Workflow
- CI/CD

### ***Related Documents***

- Development\_Guidelines.md
- Git\_Workflow.md
- CI\_CD.md

---

## **DEC-0006: Documentation Generation Before Commit**

Date: 2026-07-01

Status: Accepted

### ***Context***

Documentation changes can break generation or create inconsistent derived artifacts.

The previous development workflow did not consistently place documentation generation before commit.

### ***Decision***

When Markdown documentation changes, the documentation generation command shall complete successfully before commit:

```
```bash
python scripts/generate_docs.py
```
```

### ***Rationale***

Generation before commit detects documentation generation failures before repository integration.

### ***Alternatives Considered***

Generating documentation only after push or only in CI was considered.

It was not selected because local verification should detect generation failures before commit.

### ***Consequences***

Positive:

- earlier failure detection
- synchronized documentation workflow
- clearer Definition of Done

Negative:

- documentation changes require an additional local verification step

### ***Impact***

- Documentation
- Development Workflow
- CI/CD

## ***Related Documents***

- Development\_Guidelines.md
- Git\_Workflow.md
- Testing\_Strategy.md
- CI\_CD.md

---

# **DEC-0007: Dedicated Development Branches**

Date: 2026-07-01

Status: Accepted

## ***Context***

The previous Git workflow contained inconsistent language about optional feature branches and direct commits to `main`.

The project requires one clear normal development workflow.

## ***Decision***

Normal development shall use dedicated branches.

Direct commits to `main` are not part of the normal development workflow.

## ***Rationale***

Dedicated branches support:

- review
- CI validation
- coherent change scope
- safer repository integration

## ***Alternatives Considered***

Direct development on `main` was considered.

It was not selected as the normal workflow because it reduces review and integration control.

## ***Consequences***

Positive:

- clearer integration workflow
- stronger review boundary
- better CI alignment

Negative:

- branch management is required for routine development

## ***Impact***

- Development Workflow
- Repository Policy
- CI/CD

## ***Related Documents***

- Git\_Workflow.md
- Development\_Guidelines.md
- CI\_CD.md

---

# Maintenance

Update the Decision Log when:

- a significant product direction changes
- a significant technology choice changes
- a development-process policy changes
- a repository policy changes
- an accepted Decision Log entry becomes obsolete
- an architecture ADR should be discoverable from the central index

Minor implementation details shall not be recorded here.

---

# Decision Governance

Before creating a Decision Log entry:

1. Identify the concrete requirement.
2. Determine whether the decision is architecture-significant.
3. If architecture-significant, use an ADR.
4. Identify real affected consumers.
5. Evaluate materially relevant alternatives.
6. Evaluate consequences.
7. Draft the decision as `Proposed`.

Before accepting a Decision Log entry:

1. Review the context.
2. Review the decision.
3. Review the rationale.
4. Review alternatives.
5. Review consequences.
6. Review impact.
7. Synchronize affected documentation.
8. Update the Decision Index.
9. Set status to `Accepted`.

---

# Decision Review Checklist

Before accepting a decision verify:

- concrete requirement identified
- correct decision system selected
- architecture decisions use ADRs
- product, technology or process decision is significant
- real consumers identified
- decision explicit
- rationale project-specific
- realistic alternatives evaluated
- positive consequences documented
- negative consequences documented
- impact identified
- affected documentation synchronized
- roadmap impact evaluated where relevant
- Decision Index updated
- documentation generation passed

---

# Related Documents

- Product\_Vision.md
- Product\_Roadmap.md

- Project\_Overview.md
- Architecture.md
- Technical\_Specifications.md
- Development\_Guidelines.md
- Git\_Workflow.md
- CI\_CD.md
- AI\_Agent\_Guide.md
- AGENTS.md
- docs/adr/README.md